



Read distraction-free on Substack

Spec Driven Development - A Deep Dive

#181: How AI Agents Build Enterprise Software at Scale



NEO KIM

9 MINUTES AGO

Get my system design playbook for FREE on
newsletter signup:

Type your email...	Subscribe
--------------------	-----------

- [Share this letter](#) & I'll send you some rewards for the referrals.

AI agents can write code...

But enterprise software is where things get HARD.

Imagine you ask an AI agent to move task notifications to a new system while preserving users' existing notification preferences.

The AI agent writes code, runs tests...everything "passes".

But a user who turned off email notifications now receives an email when someone assigns them a task.

The code passes tests, but it does NOT *behave* the way you want.

A simple fix is to give the AI agent ¹ a better prompt:

- Keep current email behavior intact.



But a prompt has its limitations...

As the codebase gets bigger & agent takes on extra work, it creates 3 new *serious problems*:

1. Understand the existing system

A prompt describes what you want.

But it does NOT automatically explain how an existing system works. The agent must understand the architecture, dependencies, business logic, feature flags, APIs, data models & old behavior in the codebase (across repos).

Without this context, the agent would make a locally “correct change” that *breaks* something elsewhere...

2. Build the right change

If you add extra prompt instructions, the request becomes extremely difficult to manage.

The agent must then keep track of scope, constraints, edge cases, success criteria, dependencies across the entire task. While a vague/incomplete instruction can quickly turn into a wrong implementation.

This could multiply the mistake across many files and/or tasks...

3. Trust autonomous work

Agent autonomy = more work you trust it with.

The agent would read more code, call more tools ², change more files, and run longer workflows. So you need controls on what it can access, what it can change, where it runs, how its output gets checked, and where a human must approve the work.

Yet a “longer prompt” does NOT solve any of these problems!...

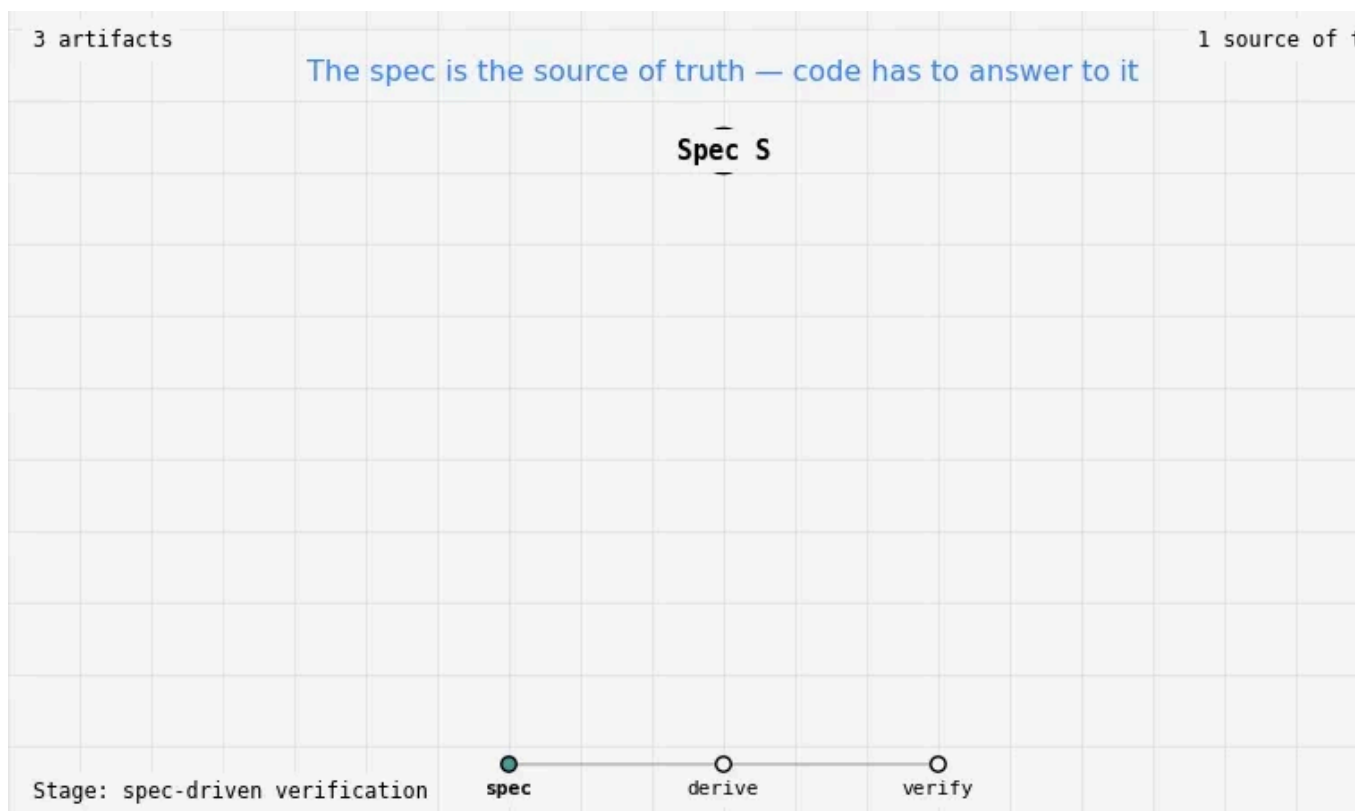
Spec-Driven Development

Spec-Driven Development (**SDD**) is a software development approach in which a specification defines intended behavior and guides planning, implementation & validation.

i.e., you define “expected” behavior, scope, constraints & success criteria *before* the agent starts to run.

Enterprise Agentic Software Development Lifecycle (Agentic **SDLC**) needs the exact same foundation:

- Full system context,
- Clear specification,
- Validation against intended result.



SDD keeps intent connected to implementation.

I'll use [Blitz](#) as the case study for this newsletter...

It's an autonomous software development platform for the enterprise that turns what you want to build into a structured plan, then autonomously executes & validates it.

i.e., you provide the requirements and intent, then review key decisions.

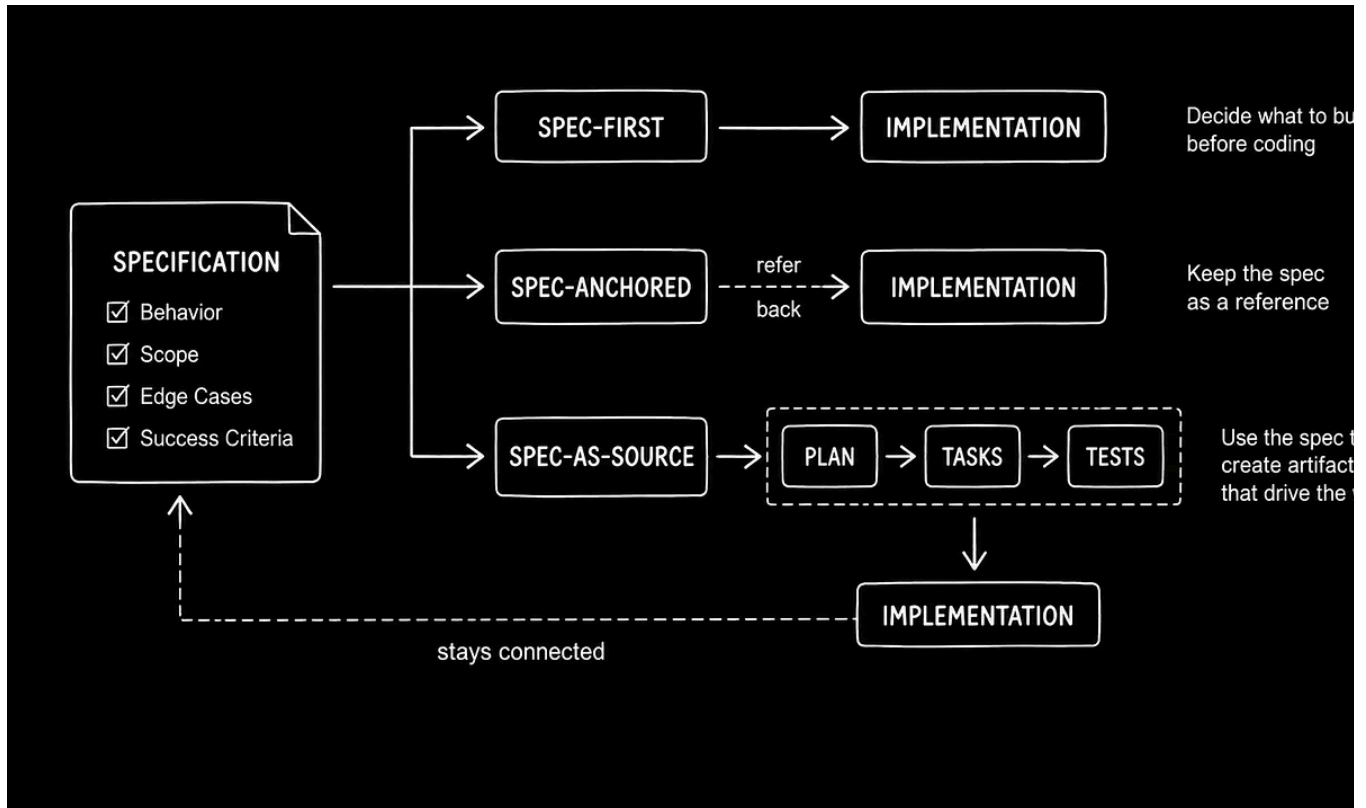
[Blitz](#) combines them with codebase context, creates the plan, executes the work with AI agents & validates the result.

Let's dive in!

How Specification Guides Development

Here are 3 roles [3](#) a specification plays in the software development lifecycle:

- **Spec-first:** You write the specification before implementation begins, so you know what to build.
 - **Spec-anchored:** You keep the specification connected to the software after implementation so it can guide future changes.
 - **Spec-as-source:** You treat the specification as the human-edited source from which you can generate and/or regenerate downstream implementation artifacts (plans, tasks, tests).
-



A specification can guide development in three different ways.

Now let's see how SDD [4](#) applies to our *notification system*...

You want to move task notifications to a new notification system. The request defines the goal, but the expected behavior still needs to be *explicit*.

So you define a specification:

- **Objective:** Move task notifications to a new system.
- **Scope:** Move task-assignment & task comment notifications. But leave billing notifications unchanged.
- **Requirements:** Preserve existing notification preferences & delivery timing. And continue using the existing email service.
- **Expected behavior:** Check the user's preferences before sending notifications. If email is enabled, queue an email through the new delivery path.
- **Edge cases:** Define what happens if delivery fails/assignee no longer belongs to the project.

- **Acceptance criteria:** Task notifications use the new system while existing use preferences continue to work correctly.

[Blitzzy](#) combines these requirements with its understanding ⁵ of the application architecture repositories.

Then it uses requirements & codebase context to create an Agent Action Plan (AAP)



Requirements + system context → Agent Action Plan.

(We'll learn how AAP works later...)

Project Rules vs Generation Prompts

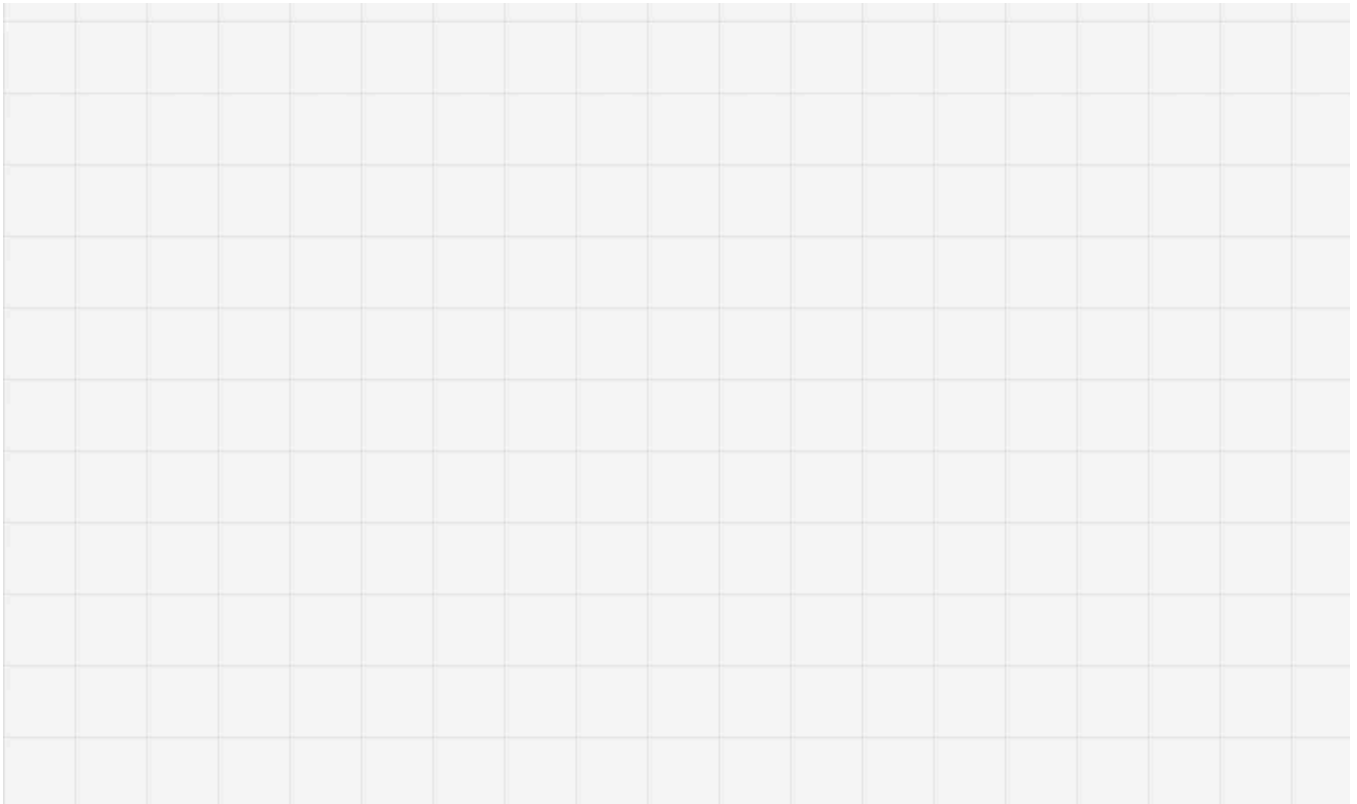
A specification defines the change itself.

Yet agents need project-level instructions ⁷ on how to do the work...

Blitzzy separates these concerns through Rules & Generation prompts:

• **Rules:** Project-wide instructions that apply whenever the agent works on the

- **Generation prompt:** Instructions for a specific change, such as its objective, scope, constraints, technical requirements & success criteria.



Rules = reusable project-level instructions; Generation prompt = specific change to implement.

Here's an example [8](#):

Rule:

Every new piece of business logic must include tests.

Generation prompt:

Move task notifications to the new system while preserving existing notification behavior and user preferences.

Requirements Clarification

A BAD assumption gets exponentially *expensive* the further it travels...

An *unclear* requirement is easier to fix while the specification is still being edited.
~~Otherwise, the agent would fill those gaps with "assumptions" and carry them into~~

So you should look for these details:

- Who will use the feature,
- How system should handle failures,
- What falls inside & outside the scope,
- Which existing services should change,
- What must remain backward compatible.

This lets you separate unknowns from assumptions:

- **Unknown:** You're missing information to make a decision.
- **Assumption:** You make a guess because information is missing.

Here's a simple clarification process for you to follow:

1. Draft the specification.
2. Identify missing/ambiguous requirements.
3. Resolve & update the specification.
4. Review the updated specification.

This process gives the agent clear requirements before it creates the implementation plan...

§

*Get 1+ referral & I'll send you my Leetcode
Master Template!*

§

1. Understand the Existing System

The agent must first understand how the *existing* system works...

i.e., it needs the correct “system context [9](#)”.

This is where Blitzy’s *Dynamic Knowledge Graph* becomes helpful.

It gives agents a global, dynamic understanding of the application across repositories... this includes architecture, dependencies & code relationships.

Here’s how it works:

Full System Context

Imagine the *specification* says:

```
Task assignment should generate an in-app notification
for the assignee, unless the user has disabled that
notification type in their preferences.
```

So the agent needs to find:

- Which components generate notifications
- Where the system stores user preferences
- Where the system handles task assignments
- What depends on the current notification flow
- Which services & APIs connect these components

... before changing the code.

And this becomes extremely HARD in large, complex existing [10](#) codebases with n repos. Relevant logic could span services, background jobs, event listeners, feature flags, data models & outdated documentation.

A larger context window is NOT enough... AI agent needs to identify & retrieve part the codebase relevant to the requested change.

1. **Analyze codebase:** Blitzzy analyzes code across multiple repositories to identify architecture, code patterns, terminology & dependencies. (You can also exclude files & directories with `.blitzzyignore`.)
2. **Build a dynamic knowledge graph:** Blitzzy maps relationships across the codebase (relationships that span repos), including control flow, call graphs, inheritance hierarchies & module dependencies.
3. **Generate a Technical Specification [11](#):** Blitzzy creates a current view of the codebase. This includes its architecture, dependencies, patterns & domain terminology.

The result is a global, dynamic understanding of the application that Blitzzy's agent can reference when planning, executing & validating changes...

Code Relationships

Retrieval-Augmented Generation (**RAG**) retrieves relevant information & adds it to model's context.

In a *codebase*, finding relevant files & components is useful, but the agent also needs to understand how they connect!..

Consider the notification system that sends an email when someone gets assigned a task... Retrieval would find [12](#):

- Preferences
- TaskAssignment
- NotificationService

You could represent their relationships using a **dynamic knowledge graph** [13](#):

Dynamic Knowledge graph [14](#) maps code components & relationships such as call dependencies, inheritance, and control flow.

Let's look at how GraphRAG uses these relationships to retrieve context

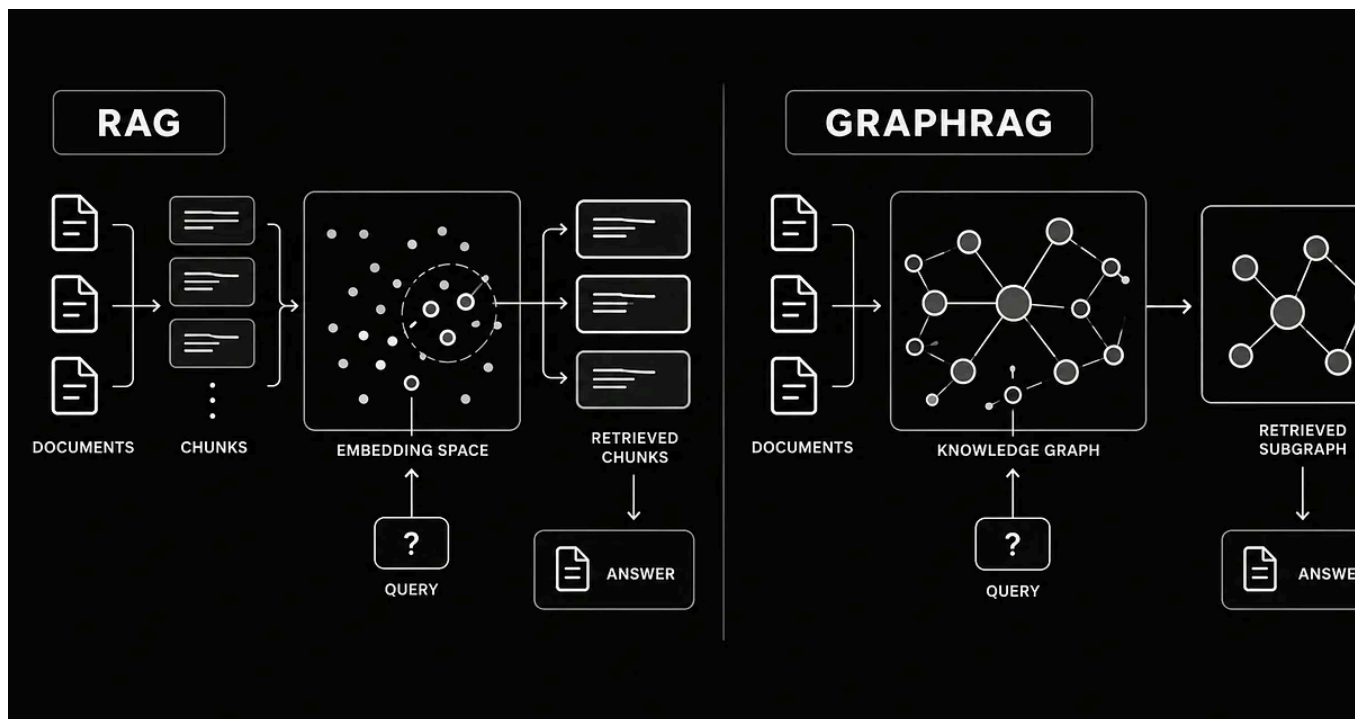
Graph Retrieval-Augmented Generation

“Relationships” become extremely useful when a single change affects *several parts* of the codebase.

Graph Retrieval-Augmented Generation (**GraphRAG**) uses a graph to retrieve content based on the relationships between pieces of information.

A traditional RAG would retrieve the relevant components...Each of them relevant on its own...

But graph-based retrieval shows “how they connect”.



RAG retrieves similar chunks; GraphRAG retrieves connected context.

If the agent needs to change what happens when “*someone gets assigned a task*”, graph could help it understand [15](#):

- What data it involves
- Which components call each other
- Which events flow through the system

In an enterprise codebase with thousands of files across many repositories, finding these connections one file at a time is extremely HARD...

A graph gives the agent a simpler way to navigate the codebase & retrieve the context [16](#) it needs.

Change Impact Analysis

Finding the correct code is only one part of the problem...

The agent must also understand the *impact* [17](#) of changing it.

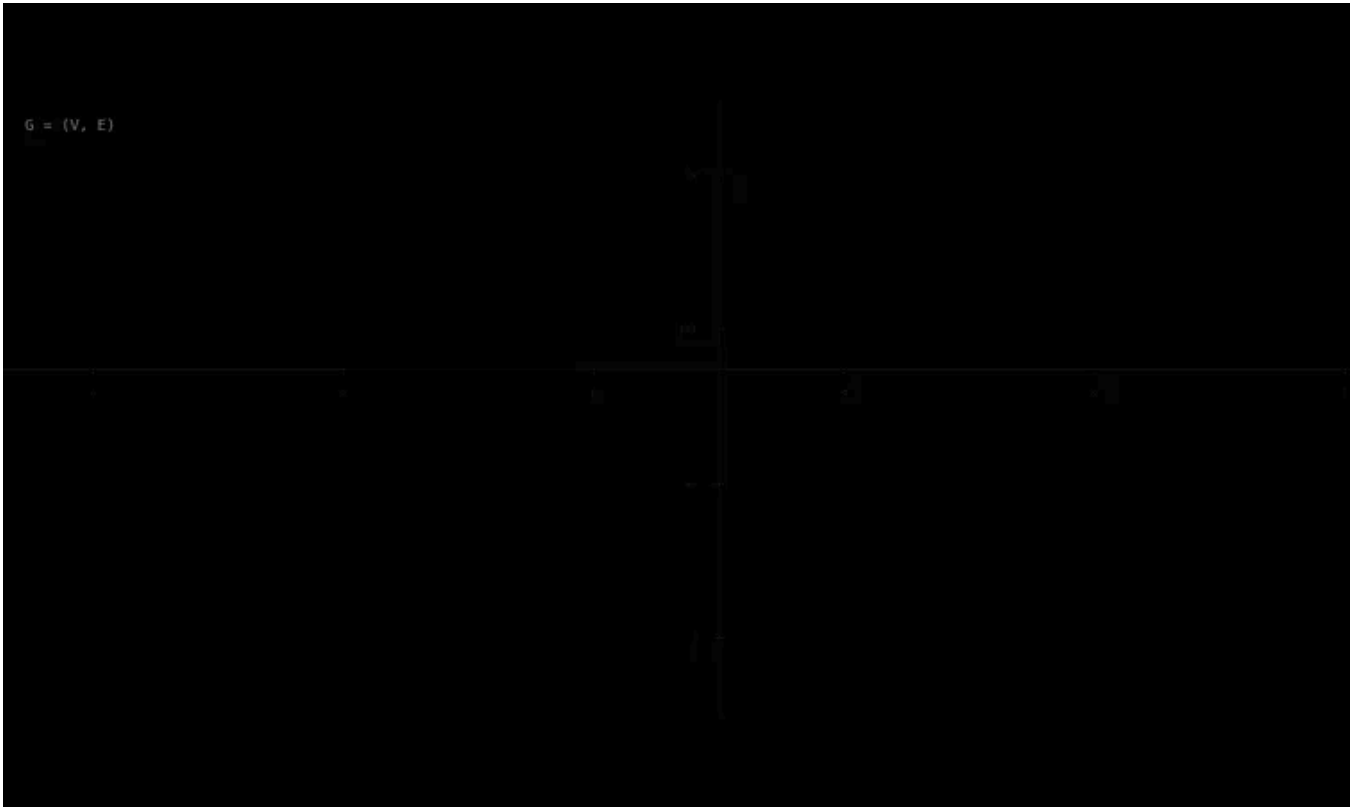


Retrieve the context that matters to the change.

Blitz's agents traverse Blitz's Dynamic Knowledge Graph in 2 directions:

- **Forward traversal** [18](#): Starts from a function/component & follows the call graph outward to understand what happens downstream.
 - **Reverse traversal** [19](#): Moves backward through the graph to find which parts of the system call depend on a function/component.
-

If the agent needs to change `NotificationService`, it can traverse forward to see what `NotificationService` calls and backward to see what calls/depends on `NotificationService`.



Traverse both directions to understand change impact.

The key is understanding how the pieces connect, NOT just what exists.

§

[Get 2+ referrals & I'll send you my Interview](#)

[Mistakes to Avoid PDF!](#)

§

2. Build the Right Change

The AI agent now needs to determine how to make the changes correctly...

ie. it needs an "implementation plan"

Ready for the best part?

Agent Action Plan

One BIG change can be broken into tiny tasks & dependencies.

For notification system migration:

- Update the data model,
- Update the delivery flow,
- Integrate with task events,

... and so on.

[Blitzzy](#) handles this with an Agent Action Plan ²⁰ (**AAP**).

It uses the generation request & global codebase context from its Dynamic Knowledge Graph to construct the AAP. The Dynamic Knowledge Graph helps map dependencies & analyze the impact of the proposed change before execution begins.

Blitzzy then algorithmically sequences the work from the AAP based on those *dependencies*. This helps agents execute the required work in a “logical order”.

You can then review & edit AAP before code generation begins.

This provides a checkpoint to catch incorrect assumptions, missing dependencies and/or unnecessary changes before they become code.

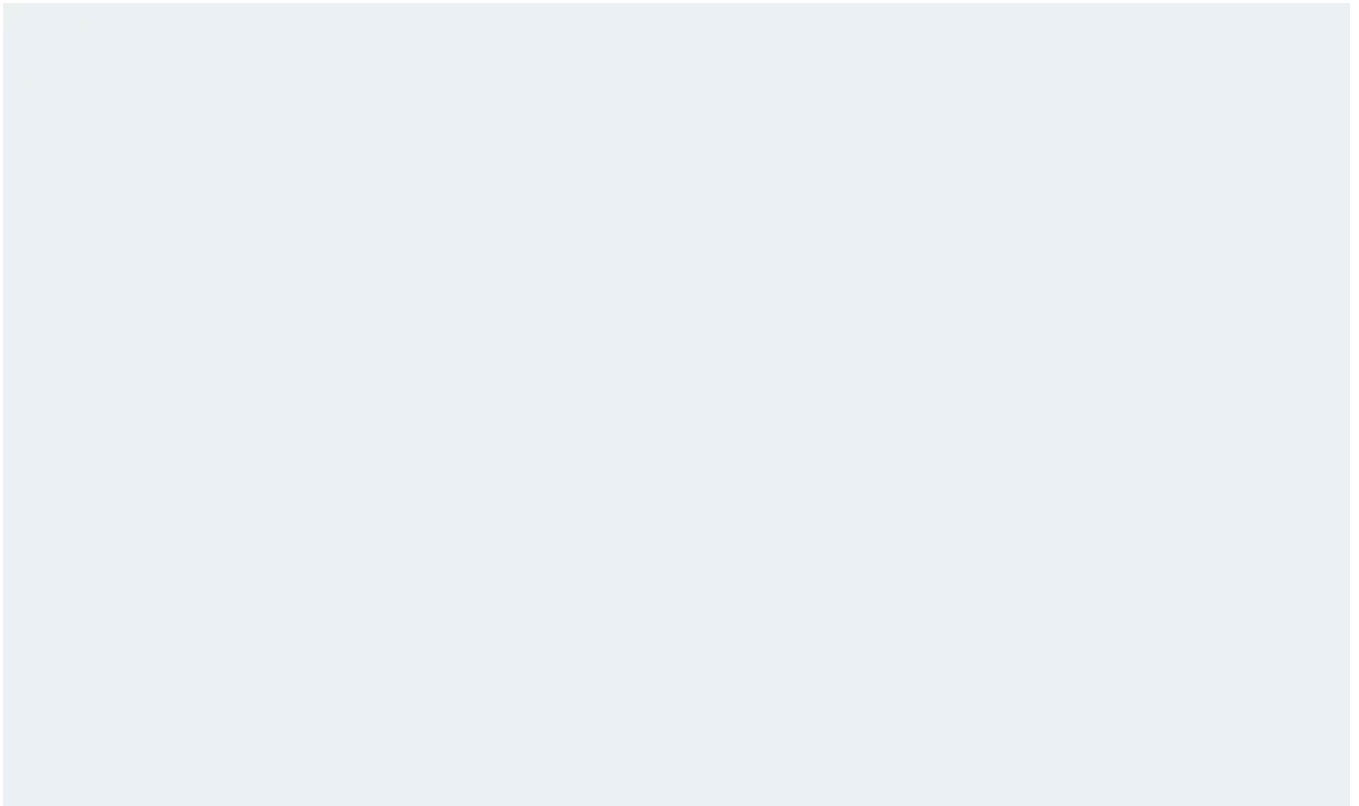
Scope and Blast Radius

AAP defines what the agent should & should NOT change.

Blast radius describes which parts of the system a change/failure would affect. For example, changing `NotificationService` could affect every component that depends on it.

- Services,
- Infrastructure,
- Files & directories,
- Components that must remain unchanged.

Blitzzy supports these boundaries through generation requests & repository exclus



Blast radius shows which parts of the system a change would affect.

This helps keep the implementation within its intended scope...

How Blitzzy Agents Execute the Plan

Once AAP is approved, Blitzzy's agents execute the planned work...

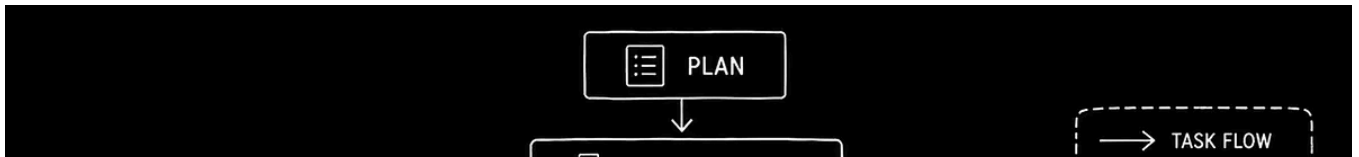
For a BIG change, Blitzzy uses AAP to *sequence* work based on "task dependencies

Then independent tasks run in parallel, while dependent tasks wait until their prerequisites finish...

Run independent work in parallel. Respect task dependencies.

Each agent needs the context & tools relevant to its task.

Also, agents need to stay aligned with requirements & codebase context so their changes work together across repositories.



An orchestrator distributes work across agents, manages dependencies & combines results for validation.

Blitzzy's orchestration layer coordinates [22](#) specialized agents across architecture, implementation, testing, debugging, validation & integration.

The resulting changes then get combined & checked as a complete implementation.

Verification and Validation

AI agent can implement the plan, pass tests, but still *miss* a “requirement.”

So the result needs to be checked against technical requirements & intended use:

- **Verification:** Check if implementation conforms to its defined requirements.
- **Validation:** Check if resulting software satisfies its intended use & user needs

Testing can occur at different levels:

- **Unit tests:** Check individual components.

Integration tests: Check how components interact with each other.

- **End-to-end tests:** Check complete workflows.
- **Acceptance tests:** Check whether software meets defined requirements & user/business needs.

But automated tests only catch behaviors they actually check...

An extra review is needed to identify missing requirements and/or problems outside test coverage...

Verification checks conformance to requirements, while validation checks whether software meets its intended use.

Blitzzy combines human oversight with automated validation throughout the development process:

- **AAP review** [24](#): You review implementation plan before code generation begins
- **Automated validation** [25](#): Blitzzy checks generated code using signals such as builds, tests, benchmarks & CI checks [26](#).
- **Project Guide & code review:** Blitzzy generates a Project Guide [27](#) with each pu

work remains. PR Review then organizes changes into logical groups, explaining why Blitzy made each change alongside the diff & surfaces relevant testing and QA results. This helps you review what changed, why it changed & how it was validated before deciding whether to merge.

i.e., passing tests alone is NOT enough...

The final implementation should be reviewed against the original requirements, scope & expected behavior.

Drift Check

As agents implement changes, code can *drift* from the original specification...

[Blitzy](#) prevents this drift during execution by orchestrating spec-adherence agents continuously check generated work against approved requirements. It validates outputs at multiple levels, from individual files through end-to-end testing.

Compare what you approved with what Blitzy built.

- Incomplete work,
- Missing requirements,
- Changes outside agreed scope,
- Remaining work that needs attention.

You make the final merge decision.

You review & test the generated changes, identify remaining gaps, and decide when to merge the pull request or request extra changes.

§

Get 3+ referrals & I'll send you Popular Interview Questions PDF!

§

3. Autonomous Work

As AI agents take on more work autonomously, you need to control what they can access, what actions they can take & when humans should step in.

Onward.

Security and Governance

Start with permissions:

- **Repository access:** Limit which code agent can access.
- **Tool access:** Limit which tools & operations it can use.
- **Network access:** Limit which external systems it can reach.
- **Least privilege:** Give AI agent only the permissions required for its task.

~~The agent also needs to distinguish trusted instructions from retrieved content~~

Code, comments & documentation can contain instruction-like text. So treat retrieved content as data, and NOT trusted instructions. This reduces prompt injection [28](#) risk

Plus controls must continue after deployment [29](#)... Monitoring helps detect *unexpected* behavior that only appears in production.

Blitzzy applies several enterprise controls across its development workflow:

- **Data security:** Customer data is encrypted in transit & at rest. Customer code is not used for model training/stored directly.
- **Isolated execution:** Code generation runs in an air-gapped environment with an inbound-only VPC architecture [30](#).
- **Deployment options:** Blitzzy supports cloud, VPC, black-box VPC, on-premises, and black-box on-premises deployments.
- **Human review:** You review the Technical Specification, approve the AAP before code generation & review generated changes before merge.
- **QA:** Multiple QA agents review generated work before delivery. Customers perform their own QA & security testing before deployment.
- **Compliance:** Blitzzy maintains SOC 2 Type II compliance & ISO 27001 certification [31](#).

These controls put boundaries around what autonomous agents can access, execute & ship...

Human Oversight

Human oversight defines when people need to review/intervene.

Here are two popular approaches:

- **Human-in-the-loop:** A person must participate at specific decision points before the agent can continue.
- **Human-on-the-loop:** The agent operates more *autonomously* while a person

A development workflow can use both...

You review specifications, approve the plan, review security-sensitive changes, & r the final release decision.

Blitzzy uses human review at key stages:

- Its Agent Action Plan creates a review point before code generation.
- After generation, automated checks use tests, benchmarks, builds, and other signals to assess the result.

As risk increases, agent autonomy decreases & human oversight increases.

Yet some changes need closer review because mistakes can have massive consequences:

- External system integrations
 - Data persistence & migrations
 - Regulatory/contractual business logic
-

These areas benefit from review by engineers, architects and/or domain experts who understand the relevant system & requirements.

§

Blitzzy SDD Workflow

You've now seen the key parts of the SDD workflow.

But how does Blitzzy bring them together into one development process?

Here's how:

1. **Codebase Onboarding:** Blitzzy analyzes the existing codebase across multiple repositories & builds a dynamic Knowledge Graph of its architecture, dependencies & relationships.
2. **Technical Specification:** Blitzzy generates a living Technical Specification to give you a view of its understanding of the existing codebase across repositories.
3. **Requirements:** You write a generation prompt to describe what you want to build/change in Blitzzy's Workspace. You define the objective, scope, technical requirements, constraints & success criteria & attach supporting files when needed.
4. **Agent Action Plan:** Blitzzy combines the generation request with *global* codebase context to create an AAP. This defines the implementation approach, scope, dependencies & required code changes.
5. **Plan review:** You review, edit & approve AAP before code generation.
6. **Multi-agent execution:** Blitzzy orchestrates specialized AI agents across different models to implement the plan. The agents work on different parts of the project.
7. **Validation:** Blitzzy orchestrates spec-adherence agents during execution & validates outputs at multiple levels, from individual files through end-to-end testing.
8. **Review handoff:** Blitzzy delivers the generated changes with a Project Guide that

9. **Human review:** You review the results, perform any remaining tasks based on next step guide & decide whether the pull request reaches production.

Thus the requirements remain connected to the implementation throughout the workflow... and help to prevent the original intent from getting lost...

§

Final Thoughts

Blitzzy shows what an SDD workflow can enable: *AI agents can take on enterprise software changes while you keep control of the outcome.*

Blitzzy connects these steps through codebase understanding, Agent Action Plans, multi-agent execution, validation & review.

You can reuse these ideas to build AI agents...

But implementing them at enterprise scale is hard. This is why I loved using Blitzzy and you should try it too.

Eligible organizations can try Blitzzy at no cost with up to 1M lines of code reverse-engineered, 25K lines of code forward-engineered, and built-in Proactive Insights to identify and prioritize risks across the software estate.

[Try Blitzzy on your codebase »](#)

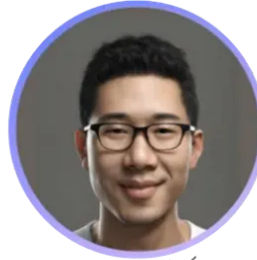
Thank you for reading.

– Neo

If you find this newsletter valuable, share it with a friend and subscribe if you have

Type your email...

Subscribe



Neo Kim

Find me on [LinkedIn](#) | [Twitter](#) | [Instagram](#)

Want to reach 250K+ tech professionals at scale? 

If your company wants to reach 250K+ tech professionals, [advertise with me](#).

Thank you for supporting this newsletter.

You are now 250,001+ readers strong, very close to 251k. Let's try to get 251k readers by 29 September. Consider sharing this letter with your friends and get rewards.

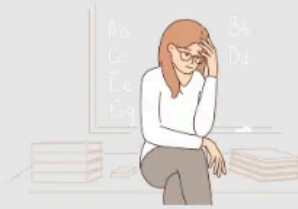
Y'all are the best.

1 Referral



Leetcode
Master
Template

2 Referrals



Interview
Mistakes
to Avoid PDF

3 Referrals



Popular
Interview
Questions PDF

References

- [Blitzzy - Introduction - Blitzzy Documentation](#)
- [Blitzzy - Tech Spec Review - Blitzzy Documentation](#)
- [Blitzzy - AAP Review - Blitzzy Documentation](#)
- [Lewis, Patrick, et al. - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks - arXiv:2005.11401](#)
- [GitHub Spec Kit - Spec-Driven Development - GitHub](#)
- [GitHub Spec Kit - Reference - GitHub Pages](#)
- [Microsoft Research - GraphRAG: Unlocking LLM Discovery on Narrative Private Data - Microsoft](#)
- [OWASP GenAI Security Project - OWASP Top 10 for LLM Applications 2025 - OWASP](#)
- [OWASP GenAI Security Project - OWASP Top 10 for Agentic Applications 2026 - OWASP](#)
- [Blitzzy - Security - Trusted AI Software - Blitzzy](#)

- [Blitzzy - Validation Checklist - Blitzzy Documentation](#)
- [Blitzzy - Project Guide Review - Blitzzy Documentation](#)
- [GitHub Spec Kit - Constitution - GitHub](#)
- [GitHub Spec Kit - Converge Command - GitHub](#)
- [Blitzzy - Quickstart - Blitzzy Documentation](#)
- [Blitzzy - System 2 AI Platform: Topping SWE-Bench Verified - Blitzzy](#)
- [Blitzzy - Blitzzy Scores a Record 84.95% on SWE-Bench Pro - Blitzzy Blog](#)
- [CXOTalk - Autonomous Software Development at Enterprise Scale: Inside a 1, Developer Pilot with Blitzzy - Episode 918](#)
- [YouTube - How Agentic SDLC Turned a 5-Month Project Into 5 Days with Brian Elliott and Tom Jackson](#)
- [Blitzzy - Taking the Long and Less-Traveled Road Is the Only Path to Autonomy Blitzzy Blog](#)
- [TipRanks - Blitzzy Showcases QAD Case Study Highlighting 3x-5x Gains in Software Development Velocity - TipRanks](#)
- [Blitzzy - Hello World! The Blitzzy C Compiler Has Arrived - Blitzzy Blog](#)
- [Blitzzy - A Quick Blitzzy Chat: 3 Codebases Take on Prompting - Blitzzy Blog](#)
- [GitHub Spec Kit - June 2026 Newsletter](#)
- [Pearce, Hammond, et al. - Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions - arXiv:2108.09293](#)
- [Blitzzy - Trust Center](#)

-
- 1 An AI agent is a software system that uses an AI model to decide what actions to take, tools & execute multi-step tasks toward a goal.
 - 2 Tool: An external capability an AI agent can call to retrieve information or perform an action.
 - 3 Think of ordering a burger at a restaurant:
-

- **Spec-anchored:** Your order remains the reference as the meal is prepared, and if the order changes.
- **Spec-as-source:** You change the order itself, and the kitchen uses the updated order to produce the meal again.

4 **What SDD Still Hasn't Solved**

SDD gives engineering teams a structured way to move from requirements to implementation, but some practical questions remain:

- **How much autonomy should an agent have?** A reversible, low-risk change may need less oversight. A change to a critical system may require tighter limits and more human review.
- **How do specifications stay aligned with code?** If the implementation changes, the specification needs to change with it. When they disagree, someone still has to decide which one reflects the intended behavior.
- **How do you provide the right context?** The information an agent needs can vary significantly from one task to another. Teams need reliable ways to provide and maintain that context.
- **How do you know SDD is working?** Possible measures include correctness, development time, rework, defects, and human review effort. The challenge is deciding which ones matter for a particular team.
- **Who owns all of this?** SDD adds work beyond writing code. Specifications, system context, validation, and governance must be maintained alongside implementation. How teams divide those responsibilities will shape how SDD works in practice.

5 The specification defines the behavior you want, but Blitzzy also needs to understand the codebase it will change...

For an existing codebase, Blitzzy first ingests the code & builds a structured representation of its architecture, dependencies, and existing behavior. It also produces a living **Technical Specification** that gives you a reviewable view of its understanding of the system.

When you submit your requirements, Blitzzy analyzes them against this codebase context.

6 **Agent Action Plan (AAP)** is Blitzzy's implementation blueprint that defines the code changes

- 7 Project-level instructions are reusable rules that apply across the project, rather than requirements for one specific feature.
 - 8 For notification migration, the generation prompt could define which task events create notifications, who receives them, which channels deliver them, and how the existing notification flow should behave during the migration.
 - 9 Shows how the existing codebase works & where the change fits.
 - 10 Brownfield development: the agent is working within an existing system rather than starting from scratch. You point an AI agent at your application's repository (or repos).
 - 11 Blitzzy uses this context to provide a queryable view of the codebase's architecture, dependencies, patterns, and conventions.
 - 12 TaskAssignment triggers an event. Another component handles the event and calls NotificationService. That service triggers EmailWorker, which checks Preferences before sending the email.
 - 13 A knowledge graph represents both:
 - What exists
 - How those things connect
 - 14 This gives the AI agent something closer to a map of the codebase than a list of files.
 - 15 This helps the agent understand the impact of a change: If it changes TaskAssignment can find which downstream components might be affected.
 - 16 Relevant components and the relationships between them.
 - 17 A component can be relevant because it calls the code being changed, consumes an event it produces, shares the same data model, or relies on its current behavior. Finding these relationships means understanding how different parts of the codebase connect, not simply reading more files.
 - 18 Blitzzy describes this as following an execution thread forward from a seed function.
 - 19 Blitzzy describes this as useful for impact analysis.
-

20 Agent Action Plan (**AAP**) is Blitzzy's implementation plan for a development task.

It defines how Blitzzy's agents will implement the requested change, including relevant changes, files, dependencies, design decisions, and scope.

- Generation request = what you want changed
- AAP = how Blitzzy plans to make that change

21 The specification can define what the agent can change & what must remain untouched.

22 The orchestration layer can dynamically plan, spawn, and coordinate specialized agents while parallelizing the Agent Action Plan into workstreams covering architecture, implementation, testing, debugging, validation, and integration.

23 **Real-World Example: BCC**

Blitzzy built *BCC*, a *Rust-based C compiler*, with 229,983 lines of Rust across 129 source files and 14 SIMD headers. The project coordinated **3,600+ agents across 127 files** and produced **2,271 passing tests**.

The example shows Blitzzy's multi-agent approach at project scale: agents work across different parts of the codebase, and testing validates the combined implementation.

24 **AAP review:** Before code generation, you review the plan for requirement coverage, scope, dependencies, design decisions, edge cases, and expected changes. An incomplete AAP can lead to incomplete code.

25 **Build & test:** After code generation, you build & test changes to verify the implementation behaves as described, rather than only checking whether the code compiles.

26 CI checks: Automated Continuous Integration checks that verify code changes build, pass tests & meet quality rules before they're merged.

27 The Project Guide makes this comparison more structured.

28 **Prompt injection:** Malicious or untrusted content that tries to manipulate an AI agent in following unintended instructions.

- 30 **Inbound-only Virtual Private Cloud (VPC) architecture:** Your system can receive approved incoming connections, but it cannot freely connect out to the internet or external systems.
- 31
- **SOC 2 Type II compliance:** Shows a company's security controls have been independently audited and found to operate effectively over a period of time.
 - **International Organization for Standardization (ISO) 27001 certification:** Shows a company follows an independently audited standard for managing information security.
- 32 *Thanks to Blitzzy for partnering on this newsletter and sharing highly valuable information.*

Subscribe to The System Design Newsletter

By Neo Kim · Thousands of paid subscribers

Download my system design playbook on newsletter signup for FREE

Subscribe

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



12 Likes · 1 Restack

← Previous

Discussion about this post

Comments

Restacks



Write a comment...

© 2026 Neo Kim · [Publisher Privacy](#)
Substack · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture
